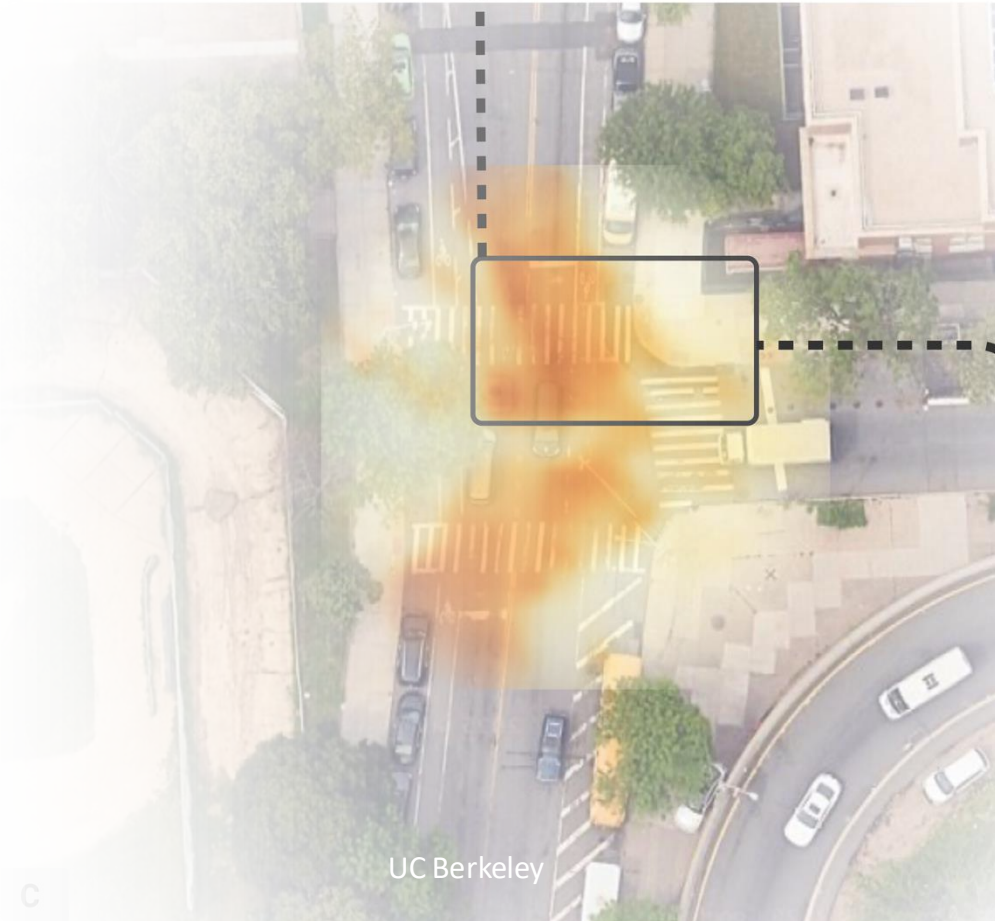
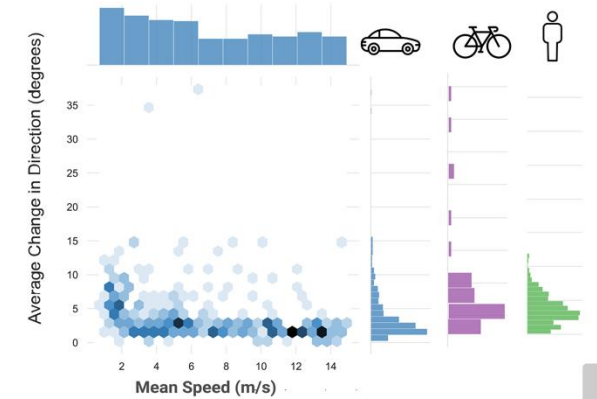
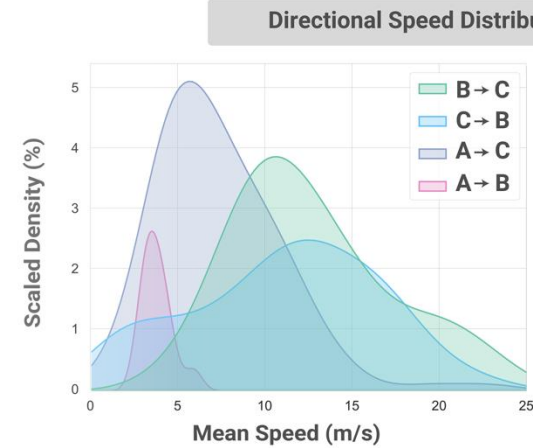
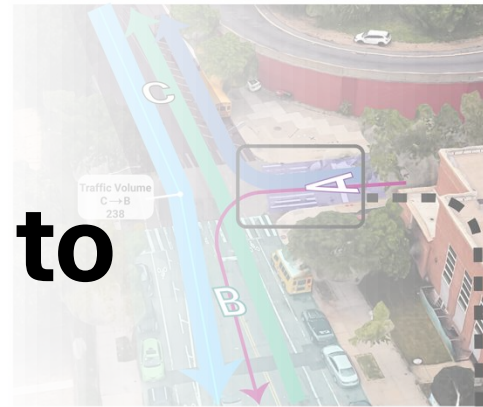


Introduction to Urban Data Analytics

Fall 2026

Maryam Hosseini

City & Regional Planning



B

D



One screwdriver per screw type

Functions are like Multi Bit Screwdrivers

- Tools designed to solve a specific problem
- Handle multiple different patterns of the same problem
- Parameters are like bits:
 - Users can choose the appropriate bit for their problem
 - Some functions come with flexible set of parameters to let you expand your choices
 - You can set default values for your function parameters



Methods are functions defined for a specific object

- Methods “fit” a specific object
- The syntax pattern in Python is:

```
object.method()  
"hello".upper()
```

- A pattern of Object Oriented Programming



Defining our Own Functions

Modularity

Decomposition

Easier Debugging

Function Definition - The Generic Format

```
def <function_name>():  
    <body>
```

A header line which begins with the keyword **def** and ends with a **colon**.

A **body** consisting of one or more Python statements, each indented the same amount
– ***4 spaces is the Python standard*** – from the header line

Function Invocation (calling)

```
def <function_name>():  
    <body>  
  
def greet():  
    print("Hello world")
```

Functions are **callable**s

calling the function

```
>>> greet()
```

```
"Hello world"
```

```
>>> greet("Max")
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: greet() takes 0 positional  
arguments but 1 was given
```

Function Invocation (calling)

It is a common mistake for beginners to forget parenthesis after the function name. Specifically, when the parameters are not required!

```
# We need () to call a callable
```

```
>>> greet
```

```
<function __main__.greet()>
```

Functions Parameters

```
def <function_name>(parameter: type) -> <return type>:  
    <body>
```

```
def greet(name: str) -> None:  
    print("Hello", name)
```

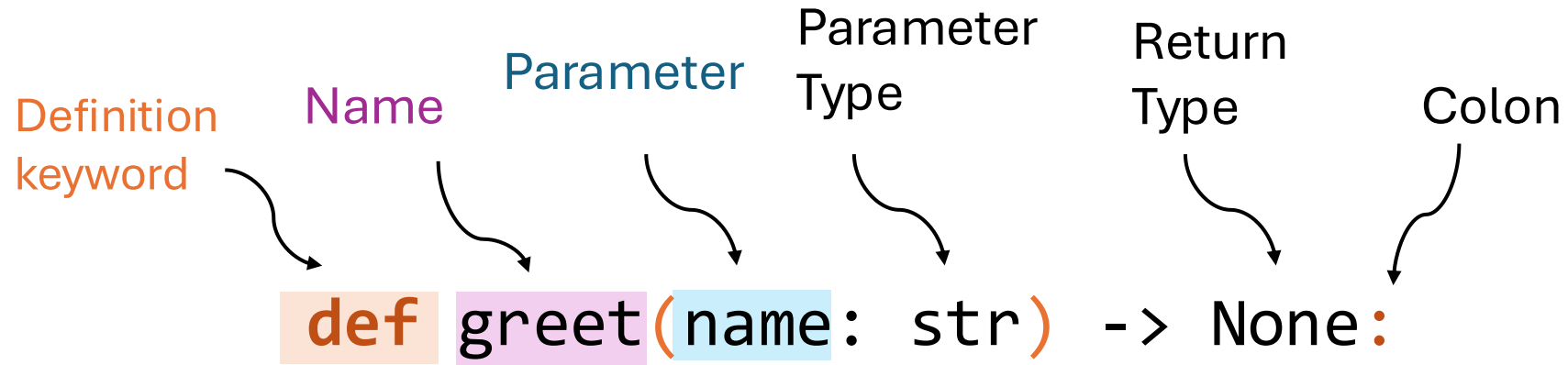
```
>>> greet("Zoe")  
"Hello Zoe"
```

```
>>> greet_var = greet("Zoe")  
print(greet_var) ? None
```

Functions

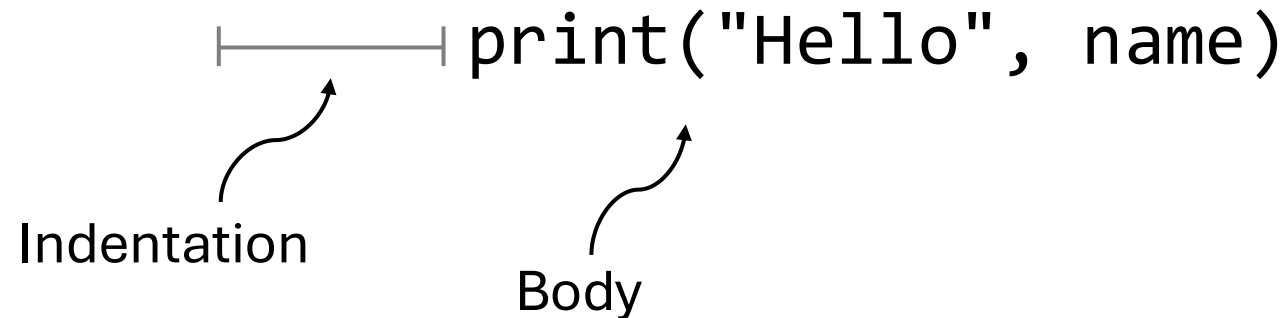
Definition keyword Name Parameter Parameter Type Return Type Colon

```
def greet(name: str) -> None:
```



```
    print("Hello", name)
```

Indentation Body



Functions: Fixed Syntax Elements

Definition keyword Parenthesis Colon

```
def greet(name: str) -> None:
```

Indentation

```
    print("Hello", name)
```

The diagram illustrates the fixed syntax elements of a Python function definition. It shows the code `def greet(name: str) -> None:` on one line and an indented line `print("Hello", name)` on the next. Arrows point from labels to specific parts of the code: 'Definition keyword' points to `def`, 'Parenthesis' points to the opening and closing parentheses of the parameter list, 'Colon' points to the colon at the end of the function signature, and 'Indentation' points to the four-space indentation of the function body line.

Functions: Fixed Syntax Elements

Part of Python's function definition syntax

- **def** (Function definition keyword)
- **()** (Parentheses for parameters)
- **:** (Colon to indicate the function body)
- **Indentation** (Required for the function body)

Functions: Customizable Elements

Can be chosen based on the function's purpose:

- **Function name:** Defines what the function does)
- **Parameter names & types:** Define inputs to the function
- **Function body:** The code that runs when the function is called)
- **Return statement & types:** What the function returns back

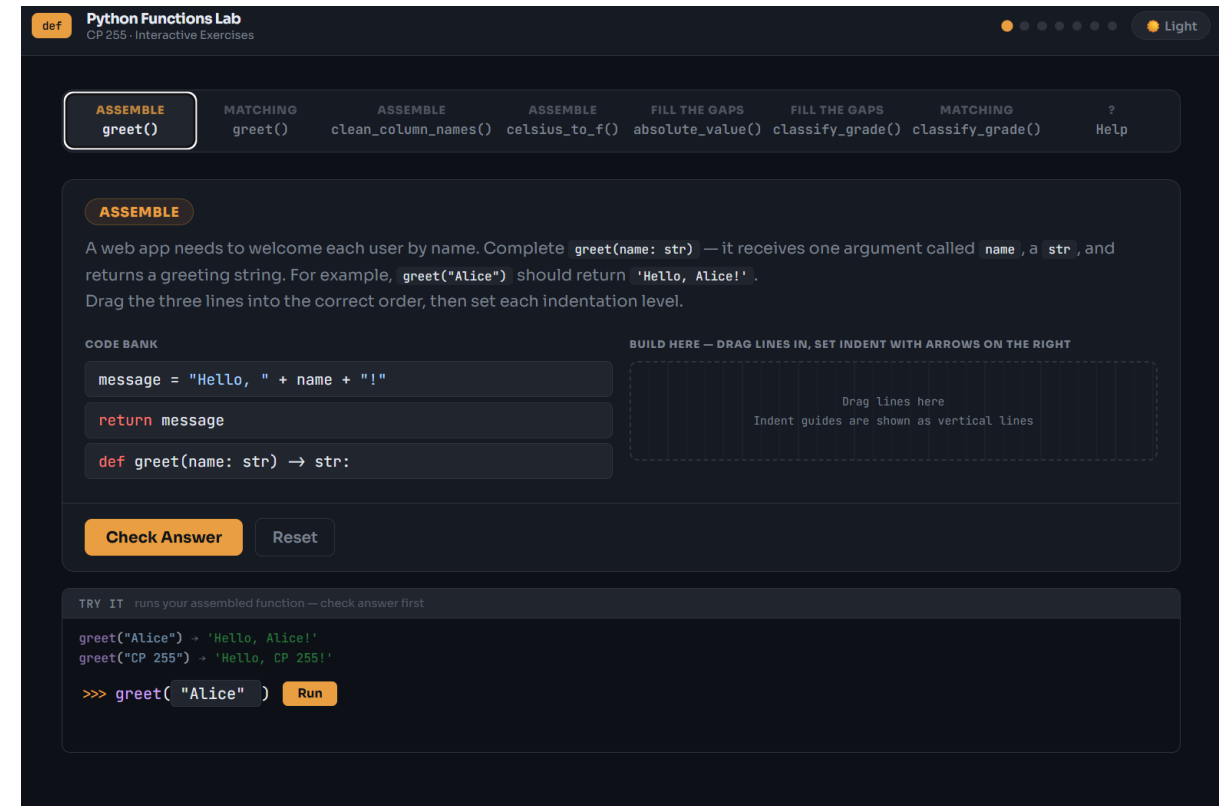
Let's Practice

10 minutes

Tools

- [Function Sandbox](#) 

I've built a [new tool](#) for you :D



The screenshot shows the 'Python Functions Lab' interface with the title 'CP 255 - Interactive Exercises'. A navigation bar at the top includes tabs for 'ASSEMBLE greet()', 'MATCHING greet()', 'ASSEMBLE clean_column_names()', 'ASSEMBLE celsius_to_f()', 'FILL THE GAPS absolute_value()', 'FILL THE GAPS classify_grade()', 'MATCHING classify_grade()', and a 'Help' link. The 'ASSEMBLE' tab is active, displaying instructions: 'A web app needs to welcome each user by name. Complete `greet(name: str)` — it receives one argument called `name`, a `str`, and returns a greeting string. For example, `greet("Alice")` should return `'Hello, Alice!'`. Drag the three lines into the correct order, then set each indentation level.'

Below the instructions is a 'CODE BANK' with three code snippets: `message = "Hello, " + name + "!"`, `return message`, and `def greet(name: str) → str:`. To the right is a 'BUILD HERE' area with vertical indent guides and the text 'Drag lines here. Indent guides are shown as vertical lines'. At the bottom of the assembly area are 'Check Answer' and 'Reset' buttons.

The bottom section, titled 'TRY IT', contains the text 'runs your assembled function — check answer first' and a code editor with the following content: `greet("Alice") → 'Hello, Alice!'`, `greet("CP 255") → 'Hello, CP 255!'`, and `>>> greet("Alice")`. A 'Run' button is next to the last line of code.

Errors & Debugging

A thick, hand-drawn style orange line that underlines the title "Errors & Debugging".

Our Brain can read through typos

"According to a research at Cambridge University, it doesn't matter in what order the letters in a word are, the only important thing is that the first and last letter be at the right place. The rest can be a total mess and you can still read it without problem. This is because the human mind does not read every letter by itself, but the word as a whole."

Computers take everything literally!

- They interpret code exactly as written, without understanding intent or context.
- **No Guesswork:** Unlike humans, computers won't infer meaning from mistakes, every symbol, space, and word must be precise.
- **Strict Rules:** Even small errors like a missing parenthesis or misaligned indentation can break the entire program.
- **Clear and Logical:** Writing proper syntax teaches you to communicate instructions clearly and logically.
- To reduce errors, master the syntax (rules) and semantics (meaning) of a programming language

Syntax Error

```
print("hello, world!")
```

```
Cell In[1], line 1  
    print("hello, world!"  
          ^
```

```
SyntaxError: incomplete input
```

Error Types



Syntax Error:

- A Python program must be fully syntactically correct before anything runs
- Syntax = the structure of code and the rules governing that structure

Runtime Error:

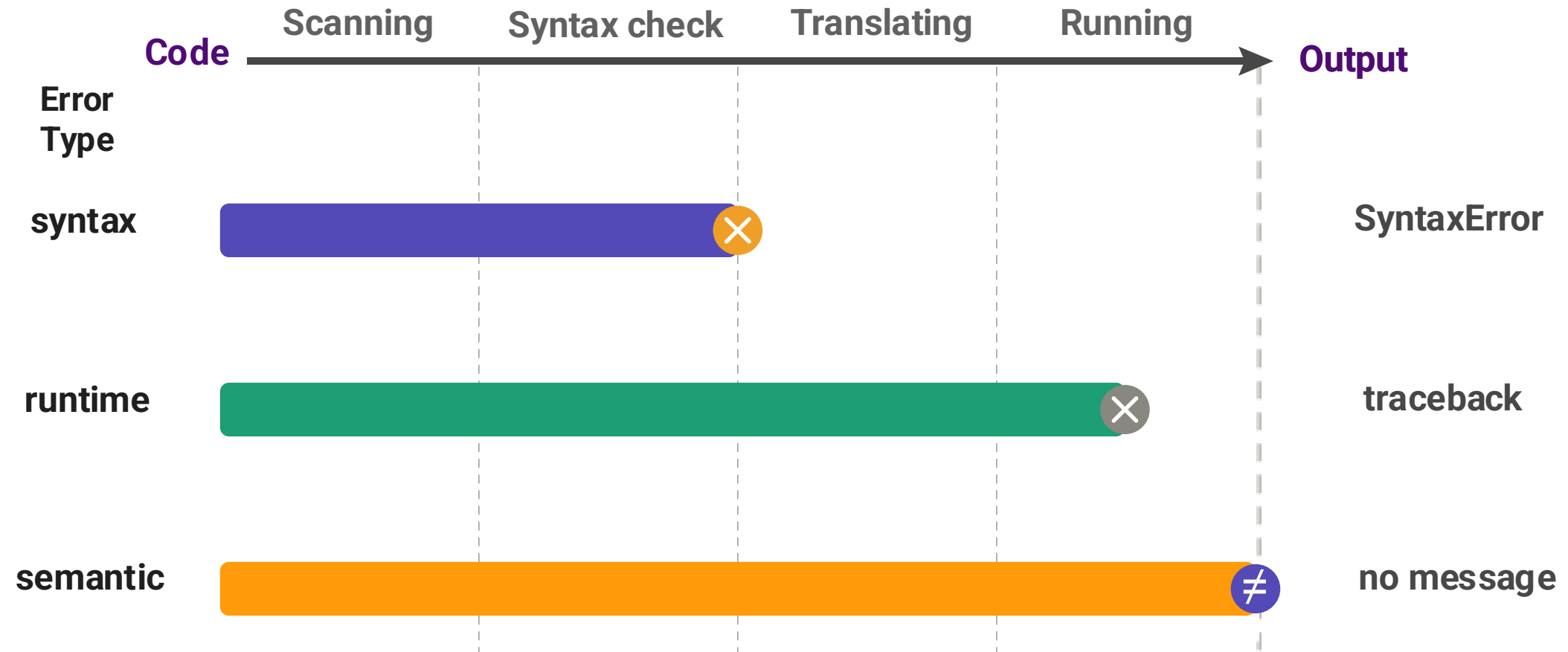
- Only appear when the program runs, syntax was fine but something broke mid-execution
- Also called **exceptions**: something unexpected happened that Python cannot recover from
- Python stops at the point of failure and prints a **traceback**

Semantic Error:

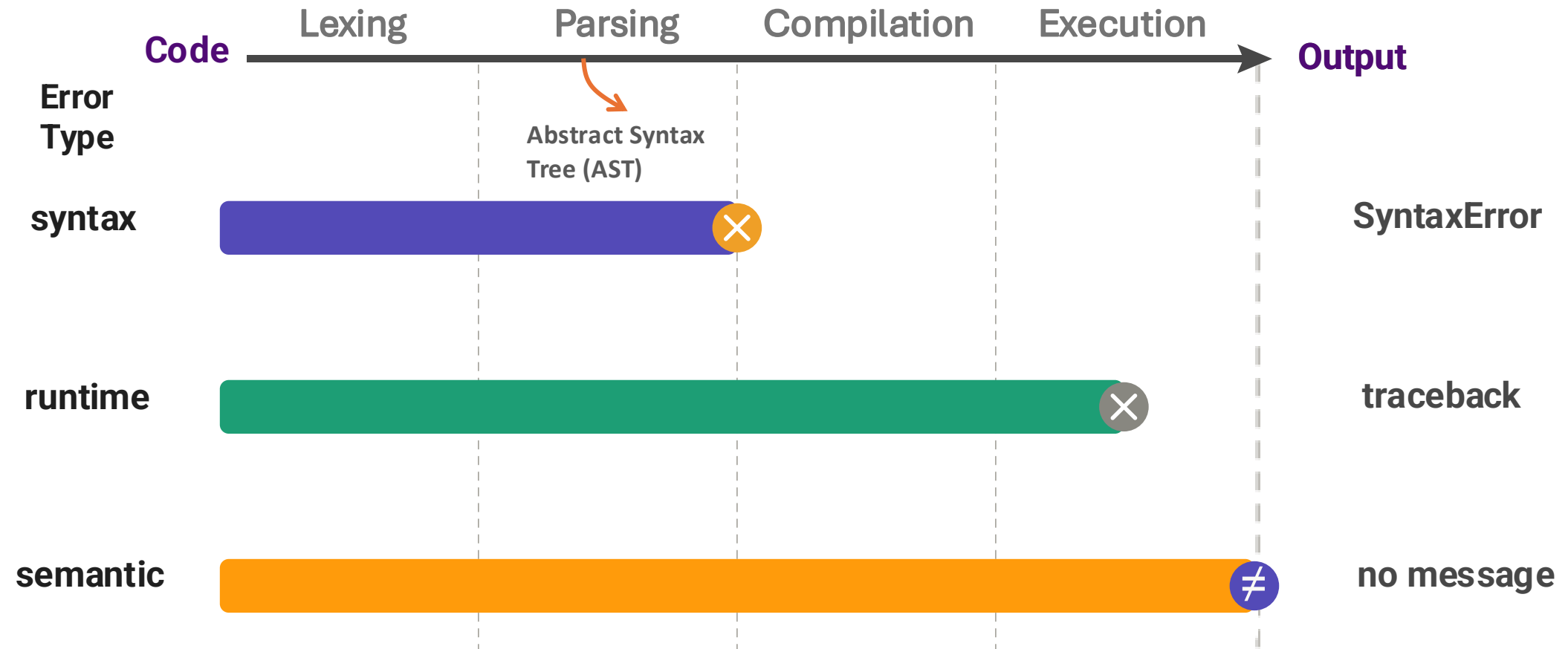
- No error messages, the program runs to completion
- Output is wrong , Python did exactly what you told it to do, not what you meant
- Hardest to find: you must work backward from the output

See: [Error types sandbox](#)

Python Error Types



Compilation (advanced)



Runtime Errors



Runtime Errors are the most common

The following list is just a small subset of the different kinds of errors:

- `NameError`
- `ImportError`
- `ValueError`
- `TypeError`
- `AttributeError`
- `IndexError`
- `...`

Tracebacks



- When Python encounters a **runtime error**, it will print out a traceback
- The message is meant to help you find **where** your error is and **what kind of error** it is
- It is called **traceback**, because it traces back through the code to show you the line number and specific line of code where the error occurred.
- Note that errors are sometimes caused by code on a previous line.

↑
read

```
Traceback (most recent call last):
```

```
-----  
File "example.py", line 9, in <module>
```

```
    summarize(berkeley)
```

```
-----  
File "example.py", line 5, in summarize
```

```
    density = get_density(city_data)
```

```
-----  
File "example.py", line 2, in get_density
```

```
    return city_data["area"]
```

```
~~~~~
```

```
KeyError: 'area'
```

```
^ start here
```

function name

which function was running

line number

where in the file

function name

called by <module>

error type

'area' key doesn't exist

Reading Tracebacks



- Tracebacks are supposed to be **read from the bottom upward**
- The very last line in a traceback is the first line that you're supposed to read:
 - Displays the **Error Type** (e.g., NameError, TypeError) &
 - A brief **Error Message** explaining what went wrong
 - That's why the first line in a traceback says **most recent call last**: the most recent call in our code (usually the one that caused the error) is the last line in the traceback.

```
Traceback (most recent call last):  
  File "example.py", line 6, in <module>  
    print(data["density"])  
KeyError: 'density'
```

Reading Tracebacks



- The last two lines on top of Error Type, show you exactly **where the error occurred**, while the lines above show the *chain of calls that led there*.
- **The error isn't always in your code:** The last lines(where the error actually triggered) might be inside a library like pandas or geopandas, not in anything you wrote.
- When that happens, don't fixate on the bottom, scan upward through the traceback until you find the last line that references your file. That's usually where the real mistake is.

Embrace the Errors!

Errors are part of the journey to becoming a better programmer

Expect them: They're how you grow!

Debugging builds problem-solving skills and confidence

Become friends with Error messages. **Read them** Read them.... **And search about them**

We don't want just any code, we want **readable, testable** code

Sandbox

5 minutes

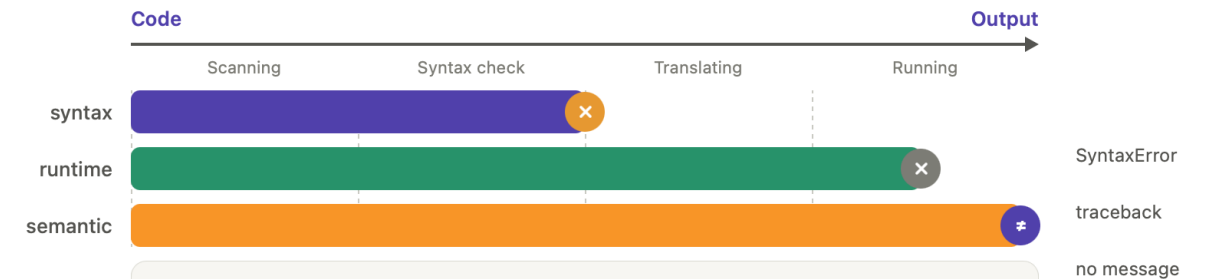
Tools

- [Function Sandbox](#) 
- [Python Error Types Lab](#) 

I've built a [new tool](#) for you :D

Python error types

Click any bar to see a code example and what Python does.



Syntax error — Python can't even read this

```
def get_density(city_data)
    return city_data["pop"] / city_data["area"]
```

```
File "example.py", line 1
    def get_density(city_data)
    ^
SyntaxError: expected ':'
```

The missing colon stops Python during the syntax check. Nothing in the file runs, not even earlier lines that were perfectly fine.

Debugging

Scientific Method ->
Rubber Duck Debugging

Debugging

The **systematic** process of

*identifying,
analyzing, &
removing*

"bugs" or errors from code to ensure it runs correctly and **produces the expected output.**

Scientific Debugging Method

1. Define the **expected behavior**

What input should the code accept, and what output should it produce?

2. Observe the failure

Run the code with a known example and compare expected vs. actual result.

3. State the bug clearly

Describe the gap: what happened instead of what should have happened?

4. Make a hypothesis

What might explain the failure?

5. Test one thing at a time

Change only one input, line, or assumption.

6. Record the result

Did the result support your hypothesis?

7. Revise and repeat

If not, undo the change and try a new hypothesis.

Rubber Duck Debugging

- 1- Get yourself a rubber duck
- 2- Inform the duck that you will be explaining your code to it
- 3- Explain what your code is supposed to do in general
- 4- Go over each line of code with the rubber duck. At some point, you will tell the rubber duck **what you are supposed to be doing next**, and you **will realize that's not actually what your code does**.

Explaining your code out loud to someone else forces you to pay closer attention to what the code does.

Just reading the code to yourself allows you to skip over mistakes. But talking to a rubber duck requires your full attention!



credit: VSCode for education

Team Proposal Workshop & Submission Practice
